

SQL Exercise Solutions

Setup

To run the code, you need to generate the data and load it into Spark tables. Use the script below to do this:

```
1 %%capture
2 %%bash
3 python ./generate_data.py
4 python ./run_ddl.py
```

```
1 %%sql --show
2 use prod.db
```

Exercise solutions

Chapter: Read data, Combine tables, & aggregate numbers to understand business performance

1. Write a query that shows the number of items returned for each region name

```
1 %%sql
2 -- identify the list of returnflag values
3 SELECT
4     l_returnflag
5 FROM
6     lineitem
7 GROUP BY
8     1
```

```
1 %%sql
2 SELECT
3     r_name AS region_name,
4     COUNT(*) AS items_returned
5 FROM
6     lineitem l
7     INNER JOIN orders o ON l_orderkey = o_orderkey
8     INNER JOIN customer c ON o_custkey = c_custkey
9     INNER JOIN nation n ON c_nationkey = n_nationkey
10    INNER JOIN region r ON n_regionkey = r_regionkey
11 WHERE
```

```

12   l_returnflag = 'R' -- R for return
13 GROUP BY
14   r_name
15 ORDER BY
16   items_returned DESC;

```

2. List the top 10 most selling parts (part name)

```

1  %%sql
2  SELECT
3     p_name AS part_name,
4     SUM(l_quantity) AS total_quantity_sold
5  FROM
6     lineitem l
7     INNER JOIN part p ON l_partkey = p_partkey
8  GROUP BY
9     p_partkey,
10    p_name
11 ORDER BY
12    total_quantity_sold DESC
13 LIMIT
14    10;

```

3. Sellers (name) who have sold at least one of the top 10 selling parts

```

1  %%sql
2  SELECT DISTINCT
3     s.s_name AS seller_name,
4     t.total_quantity_sold
5  FROM
6     lineitem l
7     INNER JOIN supplier s ON l.l_suppkey = s.s_suppkey
8     INNER JOIN (
9         SELECT
10            p_partkey,
11            p_name,
12            SUM(l_quantity) AS total_quantity_sold
13        FROM
14            lineitem l
15            INNER JOIN part p ON l.l_partkey = p.p_partkey
16        GROUP BY
17            p_partkey,

```

```

18      p_name
19    ) t ON l.l_partkey = t.p_partkey
20 ORDER BY
21      t.total_quantity_sold DESC
22 LIMIT 10;

```

4. Number of items returned for each order price bucket. The definition of order price bucket is shown below.

```

1 CASE
2   WHEN o_totalprice > 100000 THEN 'high'
3   WHEN o_totalprice BETWEEN 25000 AND 100000 THEN 'medium'
4   ELSE 'low'
5 END AS order_price_bucket

```

```

1 %%sql
2 SELECT
3     CASE
4         WHEN o_totalprice > 100000 THEN 'high'
5         WHEN o_totalprice BETWEEN 25000 AND 100000 THEN 'medium'
6         ELSE 'low'
7     END AS order_price_bucket,
8     COUNT(*) AS items_returned
9 FROM
10    lineitem l
11    INNER JOIN orders o ON l_orderkey = o_orderkey
12 WHERE
13     l_returnflag = 'R'
14 GROUP BY
15     CASE
16         WHEN o_totalprice > 100000 THEN 'high'
17         WHEN o_totalprice BETWEEN 25000 AND 100000 THEN 'medium'
18         ELSE 'low'
19     END -- Note how we can group by a case statement, not possible in all DB engines, check y
20 ORDER BY
21     items_returned DESC;

```

5. Average time (in days) between receiptdate and shipdate for each nation (name)

Round the average to 2 decimal digits

```

1  %%sql
2  SELECT
3      n_name AS nation_name,
4      ROUND(AVG(DATEDIFF (l_receiptdate, l_shipdate)), 2) AS avg_delivery_days
5  FROM
6      lineitem l
7      INNER JOIN orders o ON l_orderkey = o_orderkey
8      INNER JOIN customer c ON o_custkey = c_custkey
9      INNER JOIN nation n ON c_nationkey = n_nationkey
10 WHERE
11     l_receiptdate IS NOT NULL
12     AND l_shipdate IS NOT NULL
13 GROUP BY
14     n_name
15 ORDER BY
16     avg_delivery_days ASC
17 LIMIT
18     10;

```

Chapter: CTE (Common Table Expression) improves code readability and reduces repetition

1. Sellers (name) who have sold at least one of the top 10 selling parts (use CTE)

```

1  %%sql
2  WITH
3      top_10_parts AS (
4      SELECT
5          p_partkey,
6          p_name,
7          SUM(l_quantity) AS total_quantity_sold
8      FROM
9          lineitem l
10         INNER JOIN part p ON l_partkey = p_partkey
11     GROUP BY
12         p_partkey,
13         p_name
14     ORDER BY
15         total_quantity_sold DESC
16     LIMIT
17         10

```

```

18 )
19 SELECT DISTINCT
20     s_name AS seller_name
21 FROM
22     lineitem l
23     INNER JOIN supplier s ON l_suppkey = s_suppkey
24     INNER JOIN top_10_parts t ON l_partkey = t.p_partkey
25 LIMIT
26     10;

```

Chapter: Use window function when you need to use values from other rows to compute a value for the current row

1. Write a query on the orders table that has the following output:

1. order_month,
2. o_custkey,
3. total_price,
4. three_mo_total_price_avg
5. **consecutive_three_mo_total_price_avg**: The consecutive 3 month average of total_price for that customer. Note that this should only include months that are chronologically next to each other.

Hint: Use `CAST(strftime(o_orderdate, '%Y-%m-01') AS DATE)` to cast order_month to date format.

Hint: Use the `INTERVAL` format shown above to construct the window function to compute consecutive_three_mo_total_price_avg column.

- The orders table schema is shown below:

ORDERS (O_)	
SF*1,500,000	
ORDERKEY	
CUSTKEY	
ORDERSTATUS	
TOTALPRICE	
ORDERDATE	
ORDER-PRIORITY	
CLERK	
SHIP-PRIORITY	
COMMENT	

Figure 1: Orders table

```

1  %%sql
2  WITH
3      monthly_orders AS (
4      SELECT
5          CAST(date_format (o_orderdate, 'yyyy-MM-01') AS DATE) AS order_month,
6          o_custkey,
7          SUM(o_totalprice) AS total_price
8      FROM
9          orders
10     GROUP BY
11         CAST(date_format (o_orderdate, 'yyyy-MM-01') AS DATE),
12         o_custkey
13     )
14 SELECT
15     order_month,
16     o_custkey,
17     total_price,
18     AVG(total_price) OVER (

```

```

19     PARTITION BY
20         o_custkey
21     ORDER BY
22         order_month ROWS BETWEEN 2 PRECEDING
23         AND CURRENT ROW
24 ) AS three_mo_total_price_avg,
25 AVG(total_price) OVER (
26     PARTITION BY
27         o_custkey
28     ORDER BY
29         order_month RANGE BETWEEN INTERVAL 2 MONTHS PRECEDING
30         AND CURRENT ROW
31 ) AS consecutive_three_mo_total_price_avg
32 FROM
33     monthly_orders
34 ORDER BY
35     o_custkey,
36     order_month
37 LIMIT
38     10

```

Explanation:

1. CTE aggregates orders by month and customer, summing total prices for each customer-month combination
2. First window function (`three_mo_total_price_avg`) uses `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` for a simple 3-row rolling average
3. Second window function (`consecutive_three_mo_total_price_avg`) uses `RANGE BETWEEN INTERVAL 2 MONTHS PRECEDING AND CURRENT ROW` to ensure only chronologically consecutive months are included
4. `RANGE` with `INTERVAL` ensures gaps in months are handled correctly - if a customer has no orders in a month, that month won't be included in the average

The key difference: `ROWS` counts physical rows while `RANGE` with `INTERVAL` ensures actual chronological continuity of months.

2. From the `orders` table get the 3 lowest spending customers per day

Hint: Figure out the `PARTITION BY` column first, then the `ORDER BY` column and finally the `FUNCTION` to use to compute running average.


```

1  %%sql
2  WITH
3      daily_customer_spending AS (
4          SELECT
5              o_orderdate,
6              o_custkey,
7              SUM(o_totalprice) AS daily_spending,
8              ROW_NUMBER() OVER (
9                  PARTITION BY
10                     o_orderdate
11                     ORDER BY
12                     SUM(o_totalprice) ASC
13             ) AS spending_rank
14      FROM
15          orders
16      GROUP BY
17          o_orderdate,
18          o_custkey
19  )
20  SELECT
21      o_orderdate,
22      o_custkey,
23      daily_spending
24  FROM
25      daily_customer_spending
26  WHERE
27      spending_rank <= 3
28  ORDER BY
29      o_orderdate,
30      spending_rank
31  LIMIT
32      10;

```

Explanation:

1. CTE aggregates daily spending per customer using SUM(o_totalprice) grouped by date and customer
2. Window function breakdown:
 1. PARTITION BY o_orderdate - separate ranking for each day
 2. ORDER BY SUM(o_totalprice) ASC - rank by spending amount (lowest first)

3. ROW_NUMBER() - assigns ranks 1, 2, 3, etc. to customers. We use ROW_NUMBER to ensure we only get one rank of each, how would using RANK and DENSE_RANK change this?
3. Filter for spending_rank <= 3 to get the 3 lowest spenders per day
4. Final ordering by date and rank for clean output
5. Write a SQL query using the **orders** table that calculates the following columns:
 1. o_orderdate: From orders table
 2. o_custkey: From orders table
 3. o_totalprice: From orders table
 4. totalprice_diff: The customers current day's o_totalprice - that same customers most recent previous purchase's o_totalprice

Hint: Start by figuring out what the **PARTITION BY** column should be, then what the **ORDER BY** column should be, and then finally the function to use.

Hint: Use the LAG(column_name) ranking function to identify the prior day's revenue.

```
1  %%sql
2  SELECT
3      o_orderdate,
4      o_custkey,
5      o_totalprice,
6      o_totalprice - LAG (o_totalprice) OVER (
7          PARTITION BY
8              o_custkey
9          ORDER BY
10             o_orderdate
11         ) AS totalprice_diff
12  FROM
13      orders
14  ORDER BY
15      o_custkey,
16      o_orderdate
17  LIMIT
18      10;
```

Explanation:

1. Window function breakdown:
 1. PARTITION BY o_custkey - separate calculation for each customer

2. ORDER BY o_orderdate - chronological order of purchases for each customer
 3. LAG(o_totalprice) - gets the previous order's total price for that customer
2. totalprice_diff calculation: Current order's price minus the previous order's price using o_totalprice - LAG(o_totalprice)
 3. Result: Shows how much each customer's spending changed compared to their most recent previous purchase (NULL for first purchase per customer)